

Práce s hodnotou NULL v PostgreSQL na databázi Sakila

Příklady používají schema `public` a plně kvalifikované názvy tabulek.

Všechny příklady jsou pouze dotazy `SELECT` a nemění data.

Co znamená NULL

`NULL` neznámá nulu, prázdný text ani hodnotu `FALSE`. Znamená, že hodnota není známá, nebyla zadána nebo se na daný řádek nevztahuje.

V databázi Sakila se s `NULL` můžeme setkat například zde:

- `address.address2` je v aktuálních datech `NULL` u 603 řádků;
- `customer.email` může podle definice sloupce obsahovat `NULL`, ale v aktuálních datech jsou všechny e-maily vyplněné;
- `film.description` je v aktuálních datech `NULL` u 10 filmů;
- `film.original_language_id` je v aktuálních datech `NULL` u 954 filmů;
- `rental.return_date` je v aktuálních datech `NULL` u 182 výpůjček bez zaznamenaného data vrácení.

1. Vyhledání hodnot NULL pomocí `IS NULL`

Pro testování hodnoty `NULL` se používá `IS NULL`. Běžné porovnání pomocí `= NULL` není správné.

```
SELECT
    rental_id,
    rental_date,
    return_date
FROM public.rental
WHERE return_date IS NULL
ORDER BY rental_date;
```

Výsledkem jsou výpůjčky bez zaznamenaného data vrácení.

Nesprávný zápis by vypadal takto:

```
-- Tento dotaz nevrátí očekávané řádky.
SELECT
    rental_id,
    return_date
FROM public.rental
WHERE return_date = NULL;
```

Porovnání s `NULL` nevrací `TRUE`, ale neznámou hodnotu. Proto je potřeba použít `IS NULL`.

2. Vyhledání vyplněných hodnot pomocí **IS NOT NULL**

IS NOT NULL vybere řádky, kde hodnota existuje.

```
SELECT
    customer_id,
    first_name,
    last_name,
    email
FROM public.customer
WHERE email IS NOT NULL
ORDER BY last_name, first_name;
```

3. Nahrazení NULL pomocí **COALESCE**

COALESCE vrátí první výraz, který není **NULL**. Je vhodný pro doplnění výchozí hodnoty.

```
SELECT
    film_id,
    title,
    COALESCE(description, 'Popis není k dispozici') AS film_description
FROM public.film
ORDER BY title
LIMIT 20;
```

Pokud je **description** vyplněný, zobrazí se jeho původní hodnota. Pokud je **NULL**, zobrazí se náhradní text.

4. Více možností v **COALESCE**

Do **COALESCE** lze vložit více výrazů. PostgreSQL použije první hodnotu, která není **NULL**.

```
SELECT
    customer_id,
    CONCAT_WS(' ', first_name, last_name) AS full_name,
    COALESCE(email, 'e-mail není uveden') AS contact_email,
    COALESCE(email, CONCAT(LOWER(first_name), '.', LOWER(last_name),
        '@unknown.local'))
        AS email_or_generated_value
FROM public.customer
ORDER BY customer_id
LIMIT 20;
```

V prvním případě se při chybějícím e-mailu zobrazí informační text. Ve druhém případě se vytvoří náhradní hodnota podle jména zákazníka.

5. Rozlišení NULL a prázdného textu pomocí **NULLIF**

`NULLIF(a, b)` vrátí `NULL`, pokud jsou hodnoty `a` a `b` stejné. Jinak vrátí hodnotu `a`.

```
SELECT
    address_id,
    address2,
    NULLIF(TRIM(address2), '') AS address2_as_null
FROM public.address
ORDER BY address_id
LIMIT 20;
```

`TRIM` odstraní okrajové mezery a `NULLIF` následně převede prázdný text na `NULL`. To je užitečné při čištění dat.

6. NULL při spojování textu

Operátor `||` vrátí `NULL`, pokud je některá jeho část `NULL`. Funkce `CONCAT_WS` se chová praktičtěji, protože hodnoty `NULL` přeskočí.

```
SELECT
    customer_id,
    first_name || ' ' || last_name || ' <' || email || '>' AS
concatenation_result,
    CONCAT_WS(' ', first_name, last_name) AS full_name,
    CONCAT_WS(' ', CONCAT_WS(' ', first_name, last_name), email) AS contact_text
FROM public.customer
ORDER BY customer_id
LIMIT 20;
```

U zákazníků bez e-mailu bude `concatenation_result` celé `NULL`, zatímco `CONCAT_WS` vytvoří použitelný text bez chybějící části.

7. NULL v aritmetických výrazech

Aritmetika s `NULL` vrácí `NULL`. Pokud je například `return_date` neznámé, není možné přímo vypočítat délku výpůjčky.

```
SELECT
    rental_id,
    rental_date,
    return_date,
    return_date - rental_date AS rental_duration,
    CASE
        WHEN return_date IS NULL THEN 'výpůjčka stále trvá'
        ELSE 'výpůjčka byla vrácena'
    END AS rental_status
FROM public.rental
```

```
ORDER BY rental_date
LIMIT 20;
```

CASE zde převádí technickou hodnotu **NULL** na srozumitelný slovní stav.

8. Rozdíl mezi **COUNT(*)** a **COUNT(sloupec)**

COUNT(*) počítá všechny řádky. **COUNT(sloupec)** počítá pouze řádky, kde daný sloupec není **NULL**.

```
SELECT
    COUNT(*) AS all_customers,
    COUNT(email) AS customers_with_email,
    COUNT(*) - COUNT(email) AS customers_without_email
FROM public.customer;
```

Stejný princip lze použít například pro popisy filmů:

```
SELECT
    COUNT(*) AS all_films,
    COUNT(description) AS films_with_description,
    COUNT(*) - COUNT(description) AS films_without_description
FROM public.film;
```

9. Agregace a NULL

Agregační funkce jako **SUM**, **AVG** a **MAX** hodnoty **NULL** ignorují. Pokud jsou všechny hodnoty **NULL**, výsledek může být také **NULL**; pomocí **COALESCE** lze dodat nulu.

```
SELECT
    SUM(amount) AS total_payment,
    AVG(amount) AS average_payment,
    COALESCE(SUM(amount), 0) AS total_payment_without_null
FROM public.payment;
```

V tomto případě jsou částky vyplněné, ale **COALESCE** ukazuje obecný způsob ochrany agregačního výsledku.

10. Porovnání včetně NULL pomocí **IS DISTINCT FROM**

Operátor **IS DISTINCT FROM** porovnává hodnoty tak, aby **NULL** byl považován za běžnou porovnatelnou hodnotu. Dvě hodnoty **NULL** jsou podle tohoto operátoru stejné.

```
SELECT
    film_id,
    title,
    original_language_id,
```

```
language_id,  
original_language_id IS DISTINCT FROM language_id AS languages_are_different  
FROM public.film  
ORDER BY film_id  
LIMIT 20;
```

Opačný operátor **IS NOT DISTINCT FROM** vrací **TRUE**, pokud jsou hodnoty stejné, včetně případu, kdy jsou obě **NULL**.

11. NULL při řazení

PostgreSQL umožňuje určit, zda se mají hodnoty **NULL** při řazení zobrazit na začátku nebo na konci.

```
SELECT  
    film_id,  
    title,  
    original_language_id  
FROM public.film  
ORDER BY original_language_id NULLS LAST, title;
```

Možnosti jsou **NULLS FIRST** a **NULLS LAST**. Explicitní uvedení je vhodné, protože zlepšuje čitelnost a odstraňuje závislost na výchozím směru řazení.

12. NULL v LEFT JOIN

Při **LEFT JOIN** zůstanou zachovány všechny řádky z levé tabulky. Pokud odpovídající řádek v pravé tabulce neexistuje, její sloupce budou mít hodnotu **NULL**.

```
SELECT  
    film.film_id,  
    film.title,  
    film.original_language_id,  
    language.name AS original_language,  
    COALESCE(language.name, 'původní jazyk není uveden') AS language_label  
FROM public.film AS film  
LEFT JOIN public.language AS language  
    ON language.language_id = film.original_language_id  
ORDER BY film.title  
LIMIT 20;
```

COALESCE zde nahradí **NULL**, který vznikne buď kvůli chybějícímu **original_language_id**, nebo kvůli chybějícímu odpovídajícímu řádku.

13. Seskupení řádků s NULL

Při **GROUP BY** se všechny řádky s **NULL** ve skupinovém sloupci zařadí do jedné skupiny.

```
SELECT
    COALESCE(rating, 'bez hodnocení') AS rating_label,
    COUNT(*) AS film_count
FROM public.film
GROUP BY rating
ORDER BY rating_label;
```

V dotazu se seskupuje podle původního sloupce `rating`, nikoli podle aliasu `rating_label`. Proto se všechny hodnoty `NULL` objeví jako jedna skupina označená textem `bez hodnocení`.

Přehled nejčastějších konstrukcí

Konstrukce	Účel
<code>IS NULL</code>	vyhledání hodnot <code>NULL</code>
<code>IS NOT NULL</code>	vyhledání vyplněných hodnot
<code>COALESCE</code>	nahrazení <code>NULL</code> první dostupnou hodnotou
<code>NULLIF</code>	převedení určité hodnoty na <code>NULL</code>
<code>CASE</code>	slovní nebo jiná logika podle hodnoty
<code>COUNT(*)</code>	počet všech řádků
<code>COUNT(sloupec)</code>	počet řádků s vyplněným sloupcem
<code>IS DISTINCT FROM</code>	bezpečné porovnání včetně <code>NULL</code>
<code>NULLS FIRST, NULLS LAST</code>	umístění <code>NULL</code> při řazení
<code>LEFT JOIN</code>	zachování řádků i bez odpovídajícího záznamu

Důležitá pravidla

- `NULL` se netestuje pomocí `= NULL` ani `<> NULL`.
- Pro testování se používá `IS NULL` nebo `IS NOT NULL`.
- Výrazy s `NULL` obvykle vracejí `NULL`.
- Pro náhradní hodnotu se používá `COALESCE`.
- `COUNT(*)` a `COUNT(sloupec)` počítají různé věci.
- `NULL` není totéž co nula, prázdný řetězec nebo `FALSE`.

Při tvorbě tohoto dokumentu byl využit GitHub Copilot.